UNIVERSITY OF TEHRAN

Electrical and Computer Engineering Department

Computer Aided-Design

CA5

Name:

Mahdi Naieni 810101536

Amin Aghakasiri 810101381

## Introduction:

This project is the continuation of CA4. In this project, we are supposed to handle 3 different modes of convolution with the PE that we made in previous project.

## Project Description:

Here are the 3 modes of the convolution:

1- Mode 1




In this mode, we should multiply a row of filter into 2 rows of ifmap and then output them form psum scratchpad.

2- Mode 2



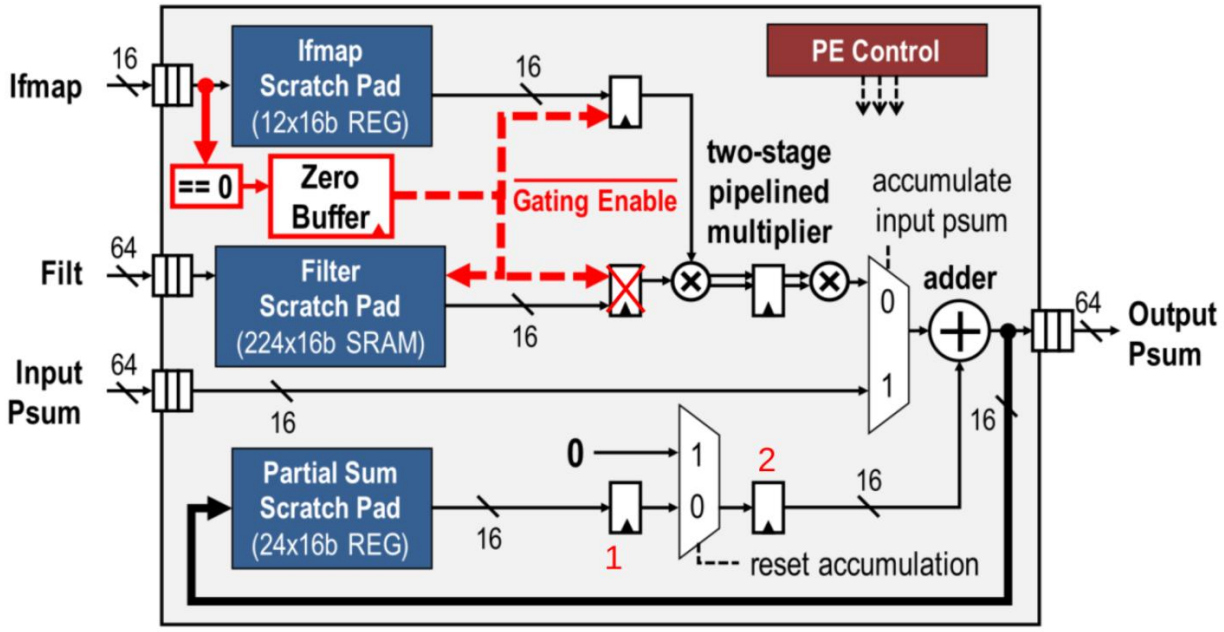

In this mode, we should multiply 2 interleaved filters into a row of ifmap and generate an interleaved output from psum.

3- Mode 3



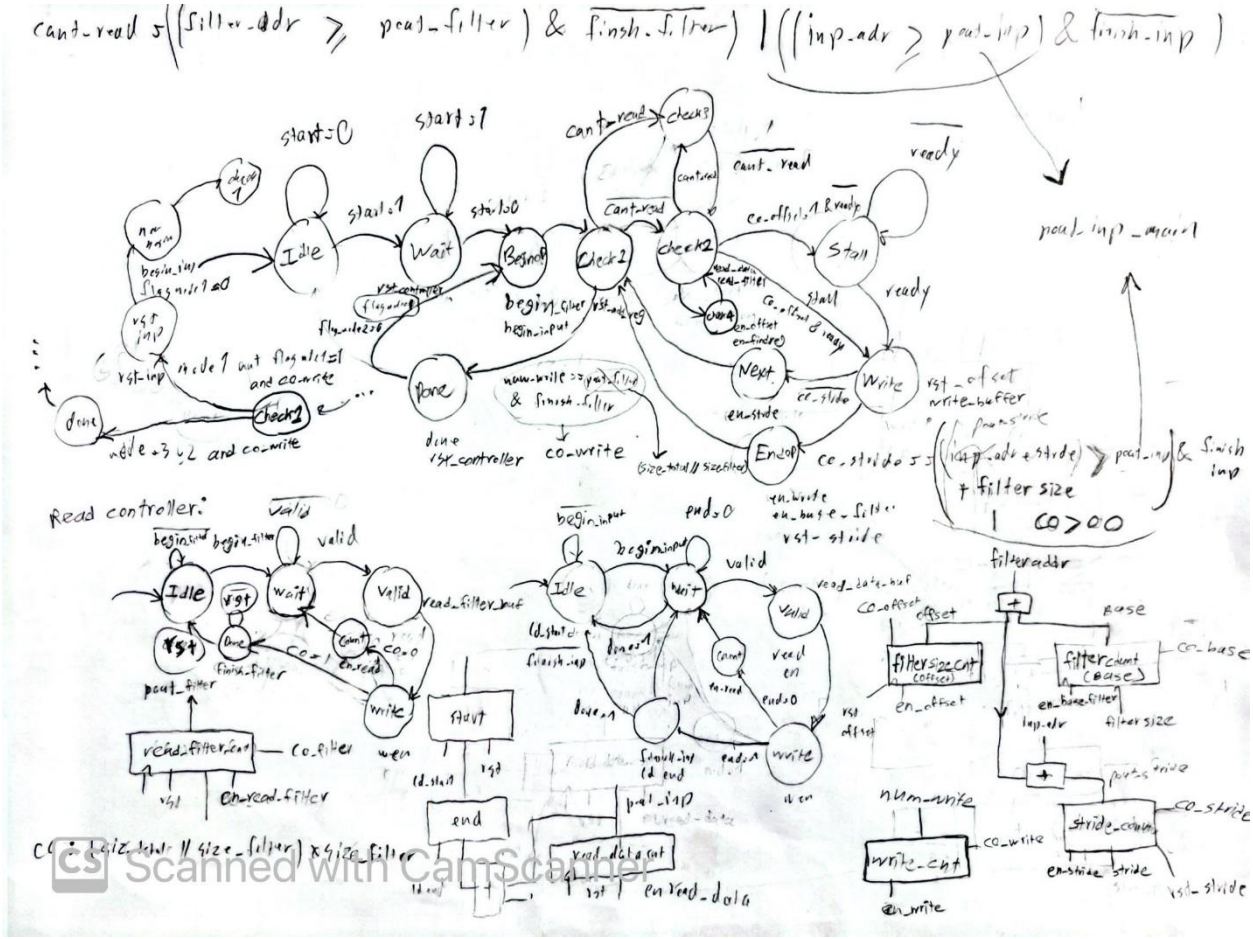

In the last mode, we should easily do a convolution of a row of filter with double size with a row of ifmap with double size.

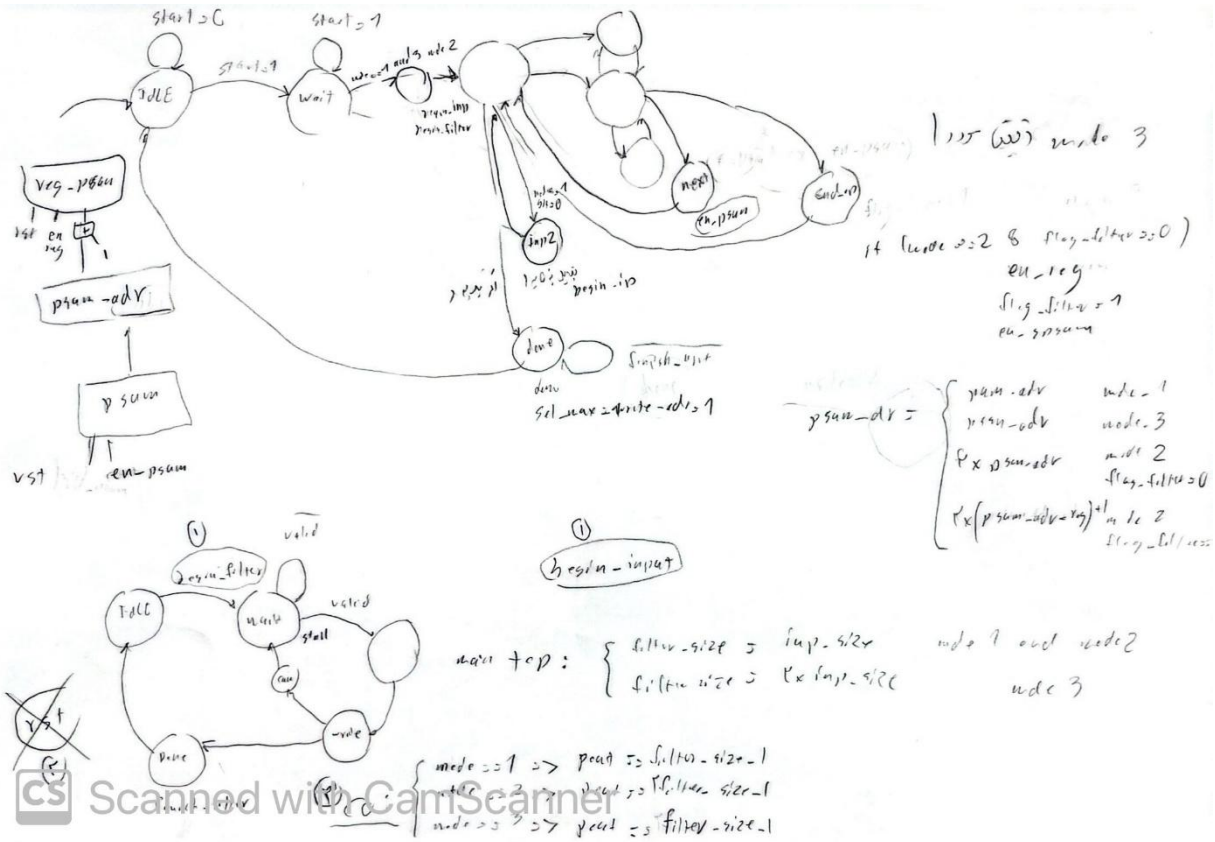
## Data path:

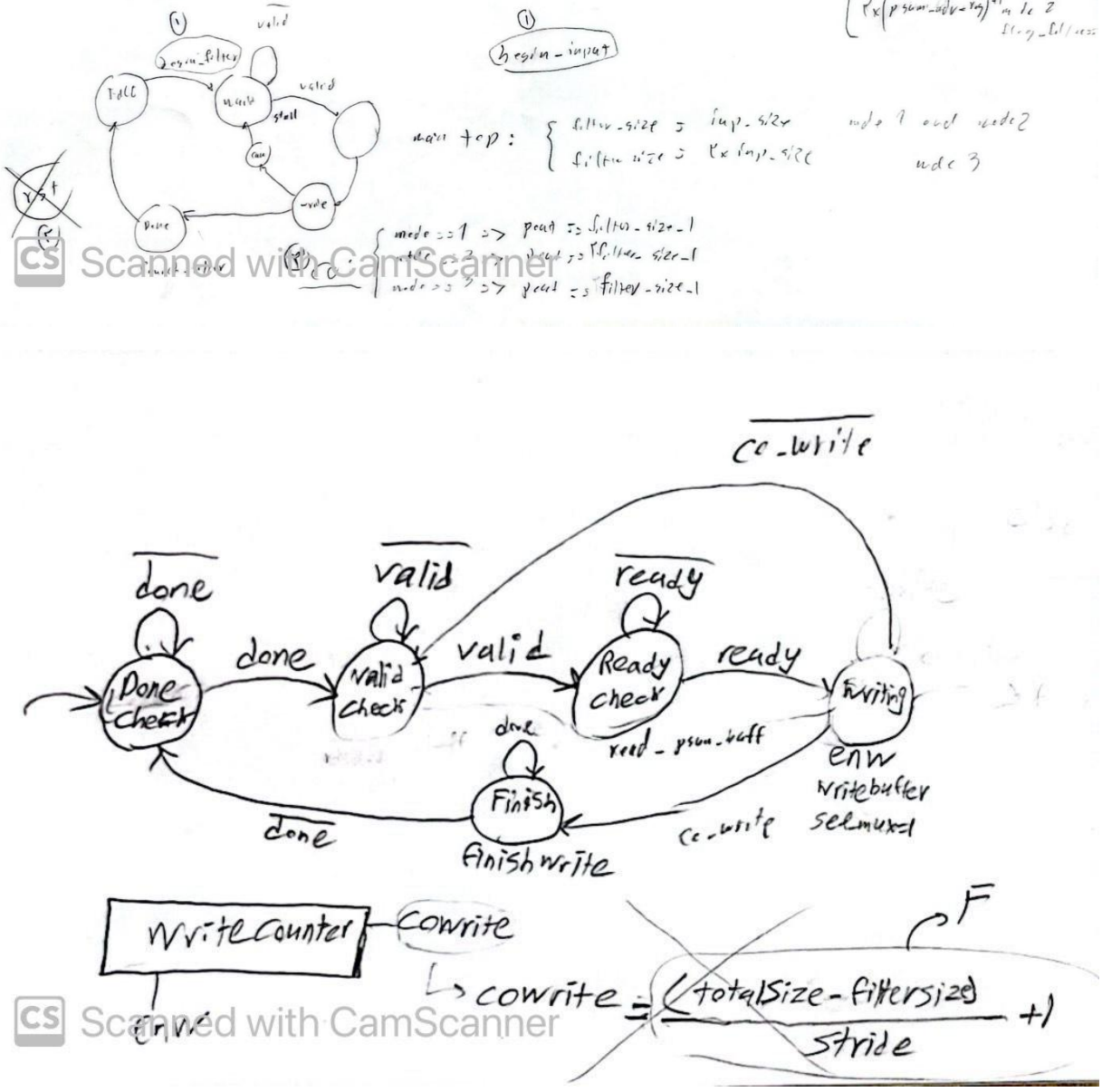



Datapath is the same as the project description.

## **Controller**:


Idle
Wait
Begin
Check1
Check2
Stall
Write
Next
Done
Endop
Read controller:
Valid
Idle
Wait
Valid
Done
Idle
Wait
Valid
Done
Wait
Write
end
filter size cnt
filter cnt
stride_cnt
write_cnt
Scanned with CamScanner

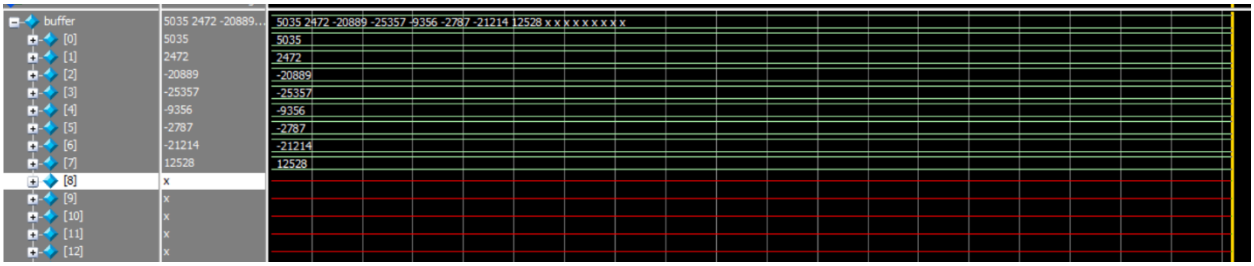


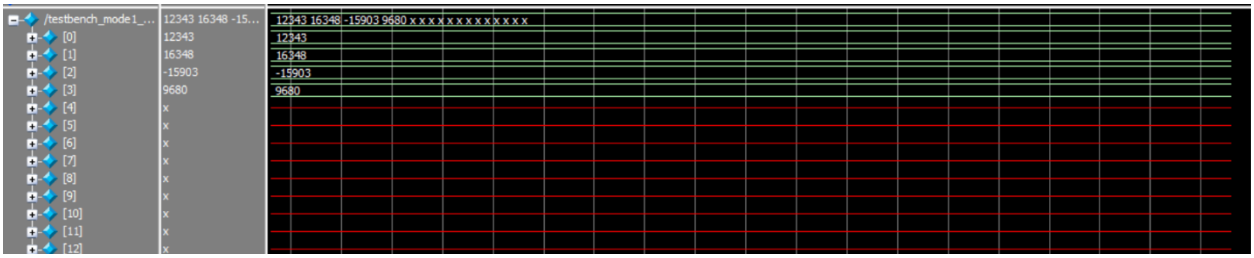

Scanned with CamScanner

We have changed the main controller a bit to handle the 3 modes. Also we have added a new scratchpad for psum. Also a controller for writing from psum scratchpad into output buffer.

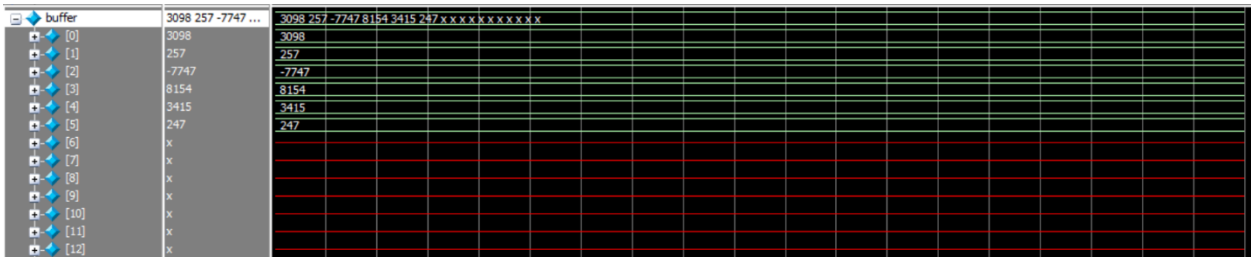
## Mode 1 Waveform (Testcase1):



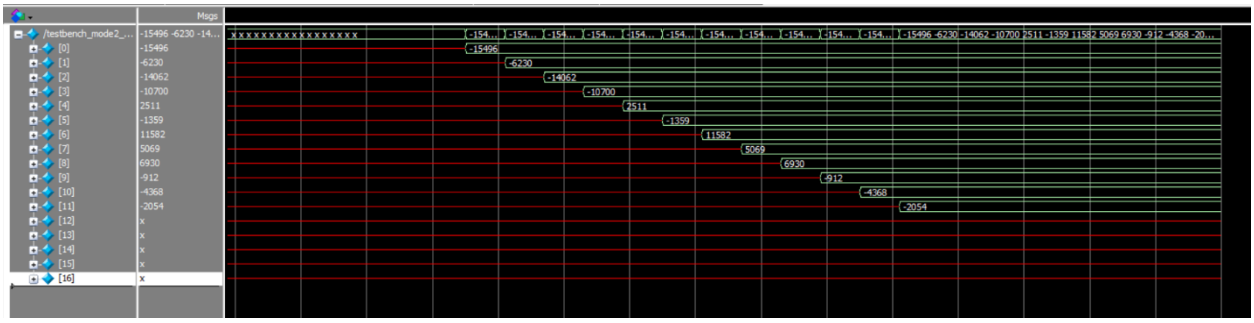

## Mode 1 Waveform (Testcase2):



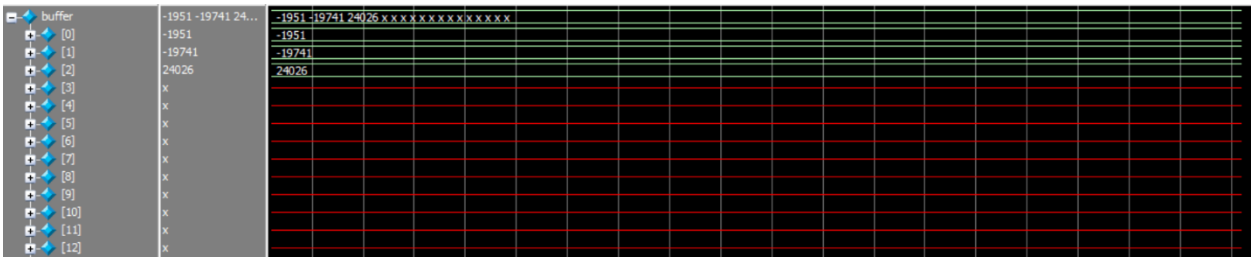

## Mode 2 Waveform (Testcase1):



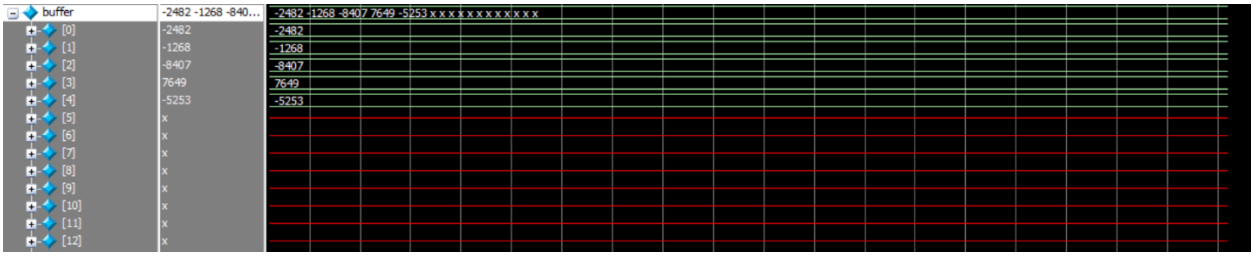

## Mode 2 Waveform (Testcase2):



## Mode 3 Waveform (Testcase 1):




## Mode 3 Waveform (Testcase 2):



# Template of Testbenches:

```
module testbench_mode1_part1 ();

    parameter DATA_WIDTH = 16, BUFFER_SIZE= 32, W_PARAM=1 , R_PARAM=1, SIZE_IFMAP= 16, SIZE_FILTER_SRAM = 16, SIZE_PSUM= 32;
    parameter ADDR_WIDTH_FILTER = $clog2(SIZE_FILTER_SRAM);
    parameter ADDR_WIDTH_IFMAP = $clog2(SIZE_IFMAP);


    reg clk=0,rst=1,start=0, write_en_ifmap=0, write_en_filter=0, write_en_psum=0;
    reg [1:0] mode = 2'b01;


    wire read_en_ifmap, read_en_filter, read_en_psum, valid_ifmap, valid_filter, valid_psum, write_en_out, ready, stall, done;

    wire [DATA_WIDTH-1:0] inp_ifmap, inp_filter, inp_psum, out_top;
    reg [DATA_WIDTH-1:0] inp_buf_filter, inp_buf_psum;

    reg [DATA_WIDTH+1:0] inp_buf_ifmap;
    wire [DATA_WIDTH+1:0] inp_ifmap_temp;

    reg [ADDR_WIDTH_FILTER-1:0] filter_size;

    reg [ADDR_WIDTH_IFMAP-1:0] stride;

    reg [2*DATA_WIDTH-1:0] mult;
    reg [DATA_WIDTH-1:0] mult_lsb;


    circular_buffer #(.DATA_WIDTH(DATA_WIDTH + 2), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_ifmap
                    (.clk(clk), .rst(rst), .write_en(write_en_ifmap), .read_en(read_en_ifmap), .inp(inp_buf_ifmap),
                    .full(), .empty(), .ready(), .valid(valid_ifmap), .data_out(inp_ifmap_temp));

    circular_buffer #(.DATA_WIDTH(DATA_WIDTH), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_filter
                    (.clk(clk), .rst(rst), .write_en(write_en_filter), .read_en(read_en_filter), .inp(inp_buf_filter),
                    .full(), .empty(), .ready(), .valid(valid_filter), .data_out(inp_filter));

    circular_buffer #(.DATA_WIDTH(DATA_WIDTH), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_psum
                    (.clk(clk), .rst(rst), .write_en(write_en_psum), .read_en(read_en_psum), .inp(inp_buf_psum),
                    .full(), .empty(), .ready(), .valid(valid_psum), .data_out(inp_psum));

    circular_buffer #(.DATA_WIDTH(DATA_WIDTH), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_out
                    (.clk(clk), .rst(rst), .write_en(write_en_out), .read_en(), .inp(out_top),
                    .full(), .empty(), .ready(ready), .valid(), .data_out());

    wire end_signal, start_signal;

    assign {start_signal, end_signal, inp_ifmap} = inp_ifmap_temp;

    top_wrapper #(
        .width(DATA_WIDTH),
        .SIZE_PSUM(SIZE_PSUM),
        .SIZE_IFMAP(SIZE_IFMAP),
        .SIZE_FILTER_SRAM(SIZE_FILTER_SRAM)
    ) my_top (
        .clk(clk), .rst(rst), .start(start), .ready(ready), .valid_ifmap(valid_ifmap), .valid_filter(valid_filter), .valid_psum(valid_psum), .end_signal(end_signal),
        .mode(mode),
        .filter_size(filter_size),
        .inp_buf_ifmap(inp_ifmap),
        .inp_buf_filter(inp_filter),
        .inp_buf_psum(inp_psum),
        .stride(stride),
        .out_buf(out_top),
        .read_en_filter_buf(read_en_filter), .read_en_psum_buf(read_en_psum), .read_en_ifmap_buf(read_en_ifmap), .write_en_buf(write_en_out), .stall(stall), .done_out(done)
    );

    always begin #10;clk=~clk; end

    initial begin
        #20 start=0; rst=0; write_en_ifmap=0; write_en_filter=0; write_en_psum=0;
        filter_size = 3;
        stride = 1;
        #20

        write_en_psum=1;
        inp_buf_psum = {DATA_WIDTH{1'b0}};
        #20
```